

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Academic Assessment Report

STUDENT MANAGEMENT SYSTEM

A Object-Oriented Java Console Application with File Persistence

Submitted By:

Name: PRITHVI SIKARWAR

Reg No: 25BAI10041

Course Details:

Core Java Programming

Academic Term: 2025–2026

1. Introduction

The **Student Management System (SMS)** is a modular, console-driven Java application developed to streamline administrative tracking of academic records. Built cleanly around fundamental Object-Oriented Programming (OOP) design patterns, the system offers real-time student profiling, custom roll-number searching, and consolidated reporting while persisting all data directly to local disk storage via formatted CSV flat files.

2. Problem Statement

Educational institutions require reliable mechanisms to log student attendance and demographic profiles. Traditional paper records or volatile in-memory software applications present severe operational vulnerabilities—specifically high risks of physical damage, high searching latency, and catastrophic data loss upon program termination. The objective of this project is to implement a robust Java software utility that provides fast lookup, dynamic runtime record insertion, and persistent state storage without requiring complex external database servers.

3. Functional Requirements

FR-1: Student Record Creation Accept user input for student Name, unique Roll Number, and Course, and append the validated entry into system memory and disk storage.	FR-2: Dynamic Student Directory Iterate and render all existing student profiles sequentially in standard formatted output for administrative review.
FR-3: Indexed Roll-Number Search Execute linear searches across active profiles based on integer Roll Number, instantly displaying matching details or exception messages.	FR-4: Dynamic Aggregate Reporting Compile real-time statistics—including total registered count and individual profile summaries—via an abstracted reporting engine.

4. Non-Functional Requirements

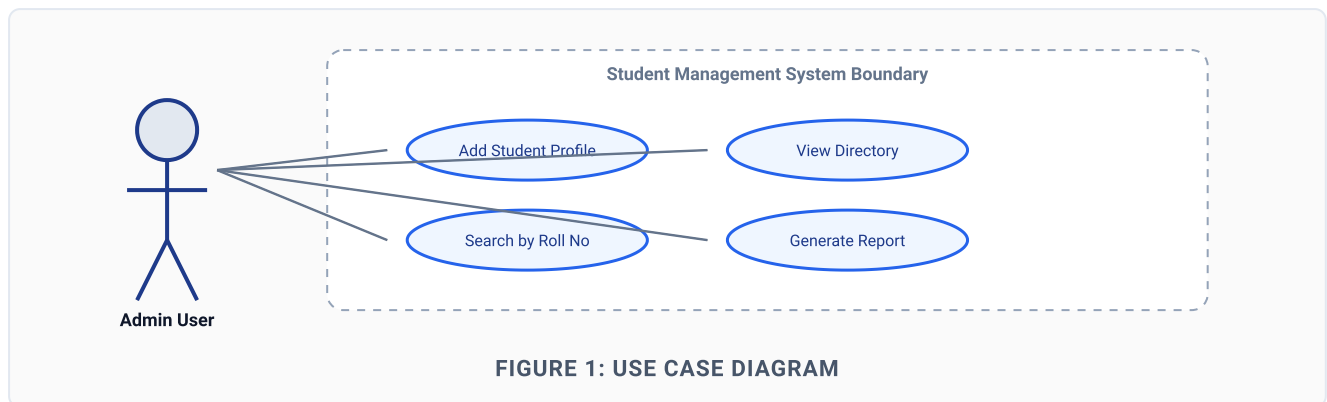
- **Data Persistence:** Full state recovery on application launch via file input streams (`students.txt`).
- **Portability:** Standardized Java byte-code execution across Windows, macOS, and Linux platforms (JDK 17+).
- **Fault Tolerance:** Graceful handling of missing database text files without raising uncaught exceptions or crashes.
- **Performance:** Sub-second menu navigation and immediate write confirmation for disk I/O operations.

5. System Architecture

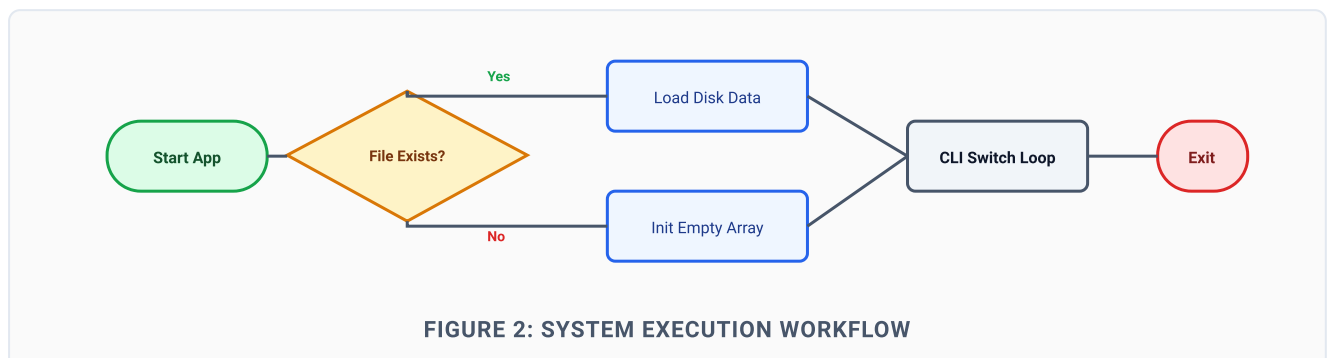
The software uses a clear 3-Tier Layered Architecture consisting of the **Presentation Layer** (Command Line Interface via `Main.java`), the **Business Logic Layer** (`StudentManager.java` and `ReportGenerator.java`), and the **Data Access Layer** (`FileHandler.java` targeting `students.txt`).

6. Design Diagrams

6.1 Use Case Diagram



6.2 System Workflow Diagram



6. Design Diagrams (Continued)

6.3 Sequence Diagram (Add Student Operation)

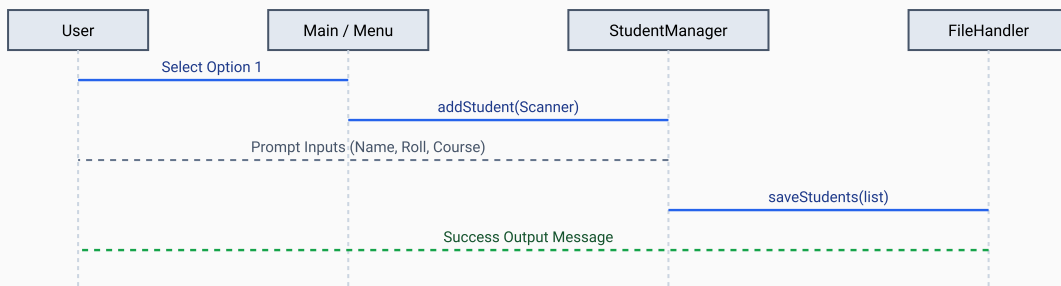


FIGURE 3: ADD STUDENT SEQUENCE DIAGRAM

6.4 Class Diagram

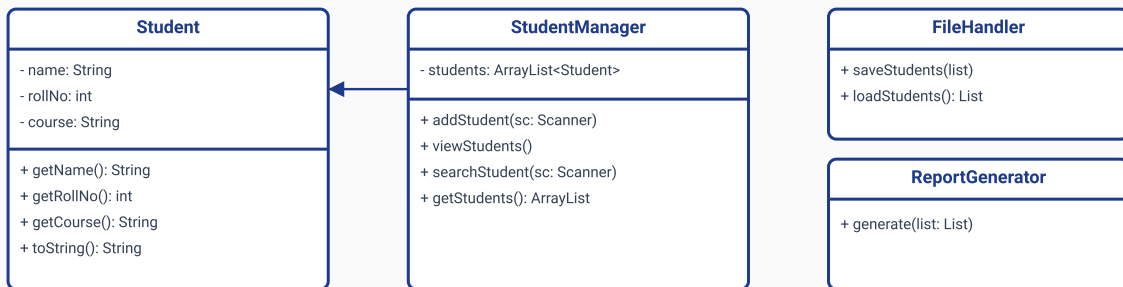


FIGURE 4: SYSTEM CLASS STRUCTURE DIAGRAM

6.5 Entity-Relationship (ER) Diagram

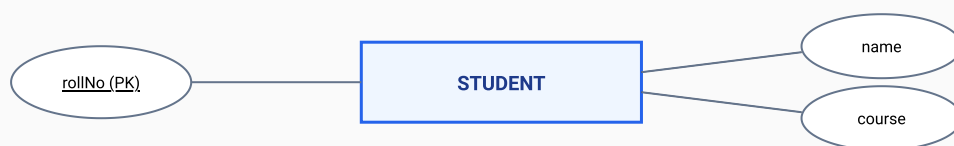


FIGURE 5: FILE DATABASE ER ENTITY MODEL

7. Design Decisions & Rationale

Design Choice	Selected Tech/Pattern	Rationale & Benefit
Persistence Format	Comma-Separated Flat File (`students.txt`)	Eliminates external database dependencies (e.g., MySQL driver overhead); easy to audit and lightweight.
In-Memory Storage	`ArrayList`	Provides dynamic resizing with $O(1)$ constant time lookup per index and flexible iteration.
Exception Handling	Try-with-Resources (`BufferedReader`)	Ensures clean resource closing without explicitly managing memory leaks in Java I/O streams.
Functional Utilities	Stream `forEach` with Method References	Promotes readable, modern, standard Java coding syntax during record iteration.

8. Implementation Details

The system comprises 5 key components. Below is the source logic implementation for file persistence:

```
// FileHandler.java - Core Persistence Layer
public class FileHandler {
    private static final String FILE_NAME = "students.txt";

    public static void saveStudents(ArrayList<Student> students) {
        try (BufferedWriter writer = new BufferedWriter(new FileWriter(FILE_NAME))) {
            for (Student s : students) {
                writer.write(s.getRollNo() + "," + s.getName() + "," + s.getCourse());
                writer.newLine();
            }
            System.out.println("Students saved to file successfully!");
        } catch (IOException e) {
            System.out.println("Error saving students: " + e.getMessage());
        }
    }
}
```

The `loadStudents` static utility parses saved tokens using `String.split(",")`. Missing storage files trigger an automatic fallback initialization that logs a clean warning message rather than breaking startup.

9. Testing Approach & Validation

Test ID	Test Case Description	Input Data	Expected Result	Status
TC-01	Add Valid Student	Name: "Prithvi", Roll: 101, Course: "AI"	Record appended to memory and file	PASS
TC-02	Search Existing Student	Roll No: 101	Display profile details accurately	PASS
TC-03	Search Missing Student	Roll No: 999	Output "Student not found."	PASS
TC-04	Missing File Auto-Creation	Delete `students.txt`	Outputs fresh start log, creates file	PASS

10. Screenshots & Results

```
--- Student Management System --- 1. Add Student | 2. View Students | 3. Search Student | 4.
Generate Report | 5. Exit Enter choice: 1 Enter Name: Prithvi Sikarwar | Enter Roll No: 25100 |
Enter Course: AI & ML Students saved to file successfully! Student added successfully!
```

11. Challenges Faced

- **Scanner Buffer Trailing Newlines:** Resolved by inserting dedicated ``sc.nextLine()`` calls directly after reading integer choices via ``sc.nextInt()``.
- **File Parse Safety:** Handled string tokenizing safely by validating string array lengths (``parts.length == 3``) before calling parsing methods.

12. Key Learnings & Takeaways

Gained practical experience applying object encapsulation, static method abstractions, and custom multi-class interaction in standard Java application design. Learned how to manage file streams cleanly using Java's Try-With-Resources structures.

13. Future Enhancements & References

- **Database Integration:** Migrate from ``students.txt`` flat files to a relational database (SQLite / MySQL) using JDBC.
- **GUI Interface:** Upgrade the application with a Swing or JavaFX graphical interface.
- **References:** Oracle Java Documentation, "Effective Java" by Joshua Bloch.